



CRYSTAL GRAPHICS DOCUMENT

Veyl

2026-06-12

1	Physics	3
1.1	Light Transport	3
1.1.1	Surface Level	3
1.1.2	Medium Level	6
1.2	Spectrum	6
2	Before Implementation	6
2.1	Light Transport Model Behavior	7
2.2	Light Transport Models	8
2.2.1	Diffuse Reflection	8
2.2.2	Specular Reflection & Refraction	9
2.2.3	Rough Reflection & Refraction	9
2.2.3.1	GGX Microfacet Model	11
2.2.4	Light Sampling	12
2.3	Multiple Importance Sampling (MIS) [1]	12
2.4	Spectrum	12
2.5	Summary	13
3	Implementation	14
3.1	Overview	14
3.2	Hit Detection	15
3.2.1	Bounding Volume Hierarchy (BVH)	15
3.2.1.1	Top Level Acceleration Structure (TLAS)	16
3.2.1.2	Bottom Level Acceleration Structure (BLAS)	16
3.3	Sample Ray Generation	17

CRYSTAL GRAPHICS DOCUMENT

3.4 GL Transmission Format (glTF)	17
3.5 Spectrum	20
Bibliography	21

1 Physics

1.1 Light Transport

This section talks about how **physics** explain the behavior of light transport on a **macro** level. Sometimes we will dive to micro levels to explain how certain patterns develop but really we only care about what we can perceive as behaviors on a macro level.

1.1.1 Surface Level

This section talks about what happens when light meets the surface of objects. Notice the object surface we talk about here can be either surface of solids, or the surface of a soap bubble. So it can be more precisely described as positions with sudden material changes.

In totality, these macro properties effect what happens when light hits the surface:

- Index of Refraction (a.k.a. IOR)
- Microgeometry (Microgeometry doesn't technically effect how light is reflected / refracted on a micro level but its characteristics determine the macro behavior of a surface.)

The micro behavior of surface light transport can be described with Snell's Law & Fresnel Equations [2]:

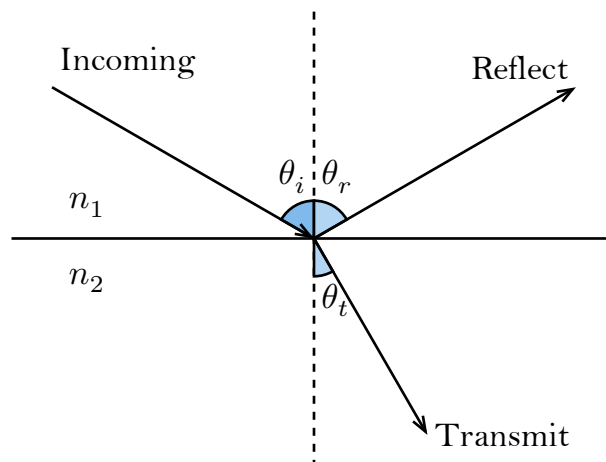


Figure 1: Micro Surface Level Light Transport

Snell's Law

$$\begin{cases} \frac{n_1}{n_2} = \frac{\sin(\theta_i)}{\sin(\theta_t)} \\ \theta_i = \theta_r \end{cases}$$

Fresnel Equations

$$\begin{cases} R_s = \left| \frac{n_1 \cos(\theta_i) - n_2 \cos(\theta_t)}{n_1 \cos(\theta_i) + n_2 \cos(\theta_t)} \right|^2, T_s = 1 - R_s \\ R_p = \left| \frac{n_1 \cos(\theta_t) - n_2 \cos(\theta_i)}{n_1 \cos(\theta_t) + n_2 \cos(\theta_i)} \right|^2, T_p = 1 - R_p \end{cases},$$

where

$$R = \frac{E_i}{E_r}, T = \frac{E_t}{E_i}$$

For **unpolarized** light,

$$R = \frac{R_s + R_p}{2}$$

It should be noted that in the case of metals, $\text{ior}(n)$ can be complex, hence the magnitude operator $\|$ in the equations.

Microgeometry

The above physics model is already enough for simulating light transport on a micro level or with perfectly smooth macro surfaces. But it fails to model situations where the surface is not perfectly smooth.

By perfectly smooth macro surfaces, we mean the surface normal has **uniform continuity**:

$$\forall \varepsilon > 0, \exists \delta > 0, (\forall \vec{x}_0, \vec{x} \in \Omega \wedge |\vec{x}_0 - \vec{x}| < \delta, |\vec{n}_0 - \vec{n}| < \varepsilon),$$

where $\Omega : F(\vec{x}) = 0$ represent the surface

In such cases, reflection & refraction can be **percieved** as having a **distribution** of directions on the macro level.

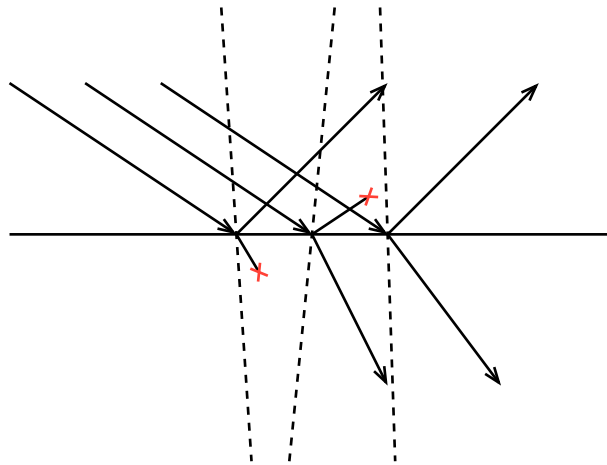


Figure 2: Rough Surface Microgeometry

The light transport behavior can be affected by the following phenomenons:

Normal Offsets

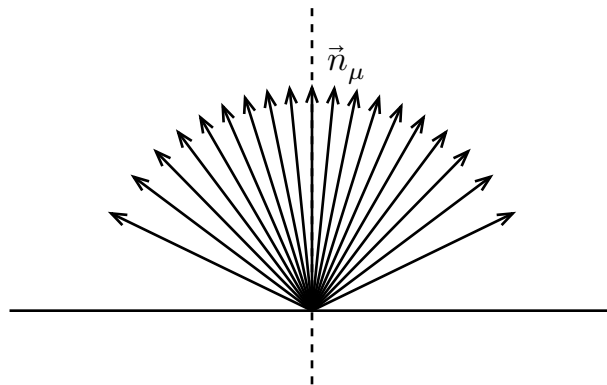
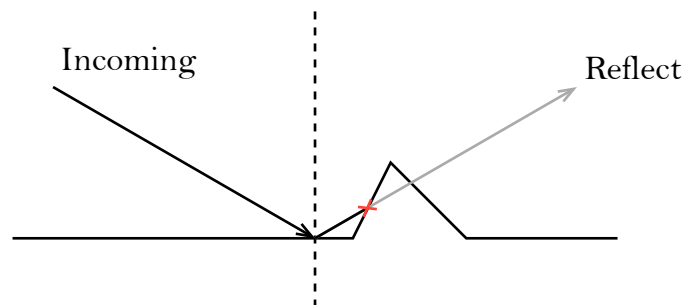


Figure 3: Normal Offsets

Masking



1.1.2 Medium Level

This section talks about what happens when light travels through some medium. For now we assume the properties of the medium doesn't change within a measurable volumn.

In the simplest scenario, when light travels through a vacume, nothing really happens. However, the complexity scales substantially when scattering & absorption happens.

1.2 Spectrum

The “percieved brightness”, a.k.a. luminance, of a spectrum can be expressed with the following integral:

$$Y = \int_{\lambda} L(\lambda) V(\lambda) d\lambda,$$

where function V denotes the spectral response curve.

For humans, which have 3 types of photoreceptor cells [3], there would be 3 response curves (except for the visually impaired).

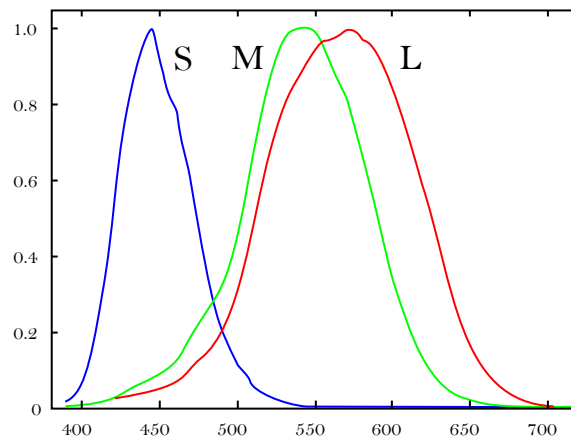


Figure 5: Human Spectral Response Curve

2 Before Implementation

This section talks about mathematical and physics tricks used in the implementation. This section *should not* contain any implementation related

information but rather some techniques/approximations that makes computing possible [4].

The Rendering Equation [5]

$$L(\vec{w}_o) = L_e(\vec{w}_o) + \int_{\vec{w}_i} L(\vec{w}_i) f(\vec{w}_o, \vec{w}_i) \cos\langle\vec{w}_i, \vec{n}\rangle d\vec{w}_i$$

2.1 Light Transport Model Behavior

Generally speaking, the outward radiance $L(\vec{w}_o)$ is a cumulative product of 2 **types** of inward radiance $L(\vec{w}_i)$:

- Continuous: Diffuse
- Discrete: Specular & Transmission

Although both types are well expressed in math with the same form (Rendering Equation), we deliberately treat them differently since this form of arithmetic expression is not easily applicable in actual computation.

Continuous Models

This type of reflection(scatter) model redirects light from a **range** of inward directions to the outward direction. As the redirection involves a continuous range, a reflection(scatter) distribution function is commonly used to approximate how much a inward direction contributes to the outward direction.

$$L_o(\vec{w}_o) = \int_{\vec{w}_i} L_i(\vec{w}_i) f(\vec{w}_o, \vec{w}_i) \cos\langle\vec{w}_i, \vec{n}\rangle d\vec{w}_i$$

Discrete Models

This type of reflection(scatter) model redirects light from certain inward angles. Since the inward angles are discrete (and also of a limited number in most cases), we can just add the elements together without the need to estimate integrals 🤖.

$$L_o(\vec{w}_o) = \sum_{\vec{w}_i} L_i(\vec{w}_i) f(\vec{w}_o, \vec{w}_i) \cos\langle\vec{w}_i, \vec{n}\rangle$$

2.2 Light Transport Models

In the real world most materials exhibit combinations of reflection & scatter behaviors, but it is easier to discuss them separately since the underlying physical process is different and we use different approximation models.

2.2.1 Diffuse Reflection

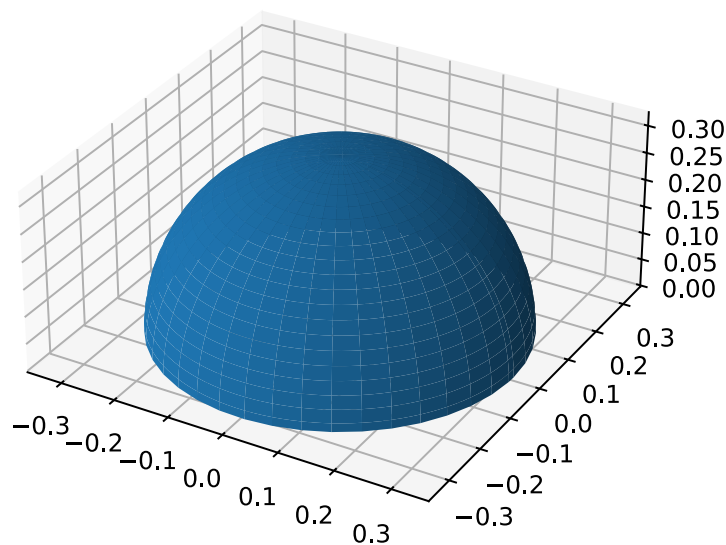


Figure 6: diffuse reflection

Lambertian Reflection

$$f = \frac{R}{\pi}$$

f is constant. And light is reflected from all inward directions in the same hemisphere.

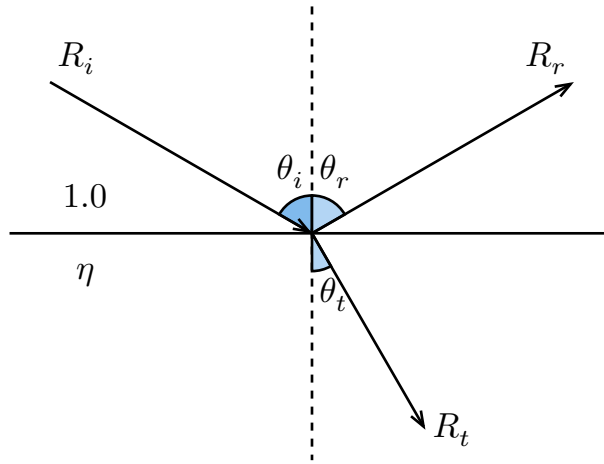
$$\text{Given } R = \iint_{\vec{w}_o} f(\vec{w}_i, \vec{w}_o) \cos \langle \vec{w}_i, \vec{w}_o \rangle d\vec{w}_o,$$

$$R = f \int_0^{2\pi} \int_0^{\frac{\pi}{2}} \cos \varphi \sin \varphi d\theta d\varphi \quad (f \equiv C)$$

$$R = f \cdot 2\pi \cdot \frac{1}{4}$$

$$f = \frac{R}{\pi}$$

2.2.2 Specular Reflection & Refraction



We can derive these functions from the fresnel-equations:

$$\begin{cases} F_r(\omega_i) = \frac{R_r}{R_i} = \frac{1}{2} \left(\left(\frac{\eta \cos \theta_i - \cos \theta_t}{\eta \cos \theta_i + \cos \theta_t} \right)^2 + \left(\frac{\cos \theta_i - \eta \cos \theta_t}{\cos \theta_i + \eta \cos \theta_t} \right)^2 \right) \\ F_t(\omega_i) = 1 - F_r \end{cases}$$

2.2.3 Rough Reflection & Refraction

The rough reflection model approximates the behavior of micro surface changes, a.k.a. microgeometry. The model is usually represented in the form of a **normal distribution function** & a **masking function**:

$$\begin{cases} D(\vec{n}_\mu) : \int_{\Omega} D(\vec{n}_\mu) d\vec{n}_\mu = p[\vec{n}_\mu \in \Omega] (\Omega \subseteq H^2) \\ G(\omega_1, \omega_2) : \begin{cases} G(\omega_1, \omega_2) = g(\omega_1)g(\omega_2) \\ \int_{H^2} D(\vec{n}_\mu) g(\omega) \max\{0, \omega \cdot \vec{n}_\mu\} d\vec{n}_\mu = \cos \theta \end{cases} \end{cases}$$

where $\vec{n}_\mu$ denotes the micro surface normal.

It should be noted that $p(\vec{n}_\mu \in \Omega)$ means the probability of some position p on the **micro surface** having a normal in the range of Ω .

Conceptually, the masking function G bridges the gap between the micro surface area and the macro surface area, owing reductions in the micro surface area to micro level blocking.

Therefore, such distributions must have the following properties:

$$\int D(\vec{n}_\mu) (\vec{n} \cdot \vec{n}_\mu) d\vec{n}_\mu = 1$$

proof:

$$\begin{aligned} |dA| &= \int_{dA} \cos \langle \vec{n}, \vec{n}_\mu \rangle dA_\mu \\ &= \int_{H^2} \cos \langle \vec{n}, \vec{n}_\mu \rangle |dA(\vec{n}_\mu)| d\vec{n}_\mu \\ &= \int_{H^2} \cos \langle \vec{n}, \vec{n}_\mu \rangle D(\vec{n}_\mu) |dA| d\vec{n}_\mu \\ &= |dA| \int_{H^2} D(\vec{n}_\mu) (\vec{n} \cdot \vec{n}_\mu) d\vec{n}_\mu \\ &\Rightarrow \int D(\vec{n}_\mu) (\vec{n} \cdot \vec{n}_\mu) d\vec{n}_\mu = 1 \end{aligned}$$

BRDF

We then derive the complete reflection distribution function:

$$\begin{aligned} f_r(\omega_o, \omega_i) &= \frac{D(\vec{n}_\mu) G(\omega_o, \omega_i) F_r(\omega_o, \omega_i)}{4 \cos \theta_i \cos \theta_o} \\ f_t(\omega_o, \omega_i) &= \left| \frac{(\omega_i \cdot \vec{n}_\mu)(\omega_o \cdot \vec{n}_\mu)}{\cos \theta_i \cos \theta_o} \right| \frac{D(\vec{n}_\mu) F_t(\omega_i) G(\omega_o, \omega_i)}{\left(\omega_o \cdot \vec{n}_\mu + \frac{\omega_i \cdot \vec{n}_\mu}{\eta} \right)^2} \end{aligned}$$

PDF

We can also combine the 2 equations to 1 **visible normal distribution** function. This function is used to sample normals given the input angle so that the reflected ray is guaranteed to be visible:

$$D(\vec{n}_\mu | \omega) = \frac{D(\vec{n}_\mu)g(\omega) \max\{0, \vec{n}_\mu \cdot \omega\}}{\cos \theta}$$

Then the reflected & refracted PDF:

$$p_r(w_o) = D(\vec{n}_\mu | \omega_i) \frac{d\vec{n}_\mu}{d\omega_o} = \frac{D(\vec{n}_\mu | \omega_i)}{4(\omega_i \cdot \vec{n}_\mu)}$$

$$p_t(w_o) = D(\vec{n}_\mu | \omega_i) \frac{d\vec{n}_\mu}{d\omega_o} = \frac{D(\vec{n}_\mu | \omega_i) |\omega_i \cdot \vec{n}_\mu|}{\left(\omega_o \cdot \vec{n}_\mu + \frac{\omega_i \cdot \vec{n}_\mu}{\eta}\right)^2}$$

2.2.3.1 GGX Microfacet Model

The GGX (Ground Glass Unknown) model uses this equation as the **normal distribution function** (Trowbridge-Reitz) [6], [7]:

$$D(\vec{n}_\mu) = \frac{1}{\pi \alpha_x \alpha_y \cos^4 \theta_\mu \left(1 + \tan^2 \theta_\mu \left(\frac{\cos^2 \varphi_\mu}{\alpha_x^2} + \frac{\sin^2 \varphi_\mu}{\alpha_y^2}\right)\right)^2}$$

Parameters

- $\alpha_x \in [0, 1]$: roughness along the x axis
- $\alpha_y \in [0, 1]$: roughness along the y axis
- θ_μ, φ_μ : inward angle

As well as a **masking function** on the assumption that $G(\omega, \omega_\mu) \equiv G(\omega)$ (Smith's approximation) [8]:

$$\int_{H^2} D(\vec{n}_\mu)g(\omega, \vec{n}_\mu) \max\{0, \omega \cdot \vec{n}_\mu\} d\vec{n}_\mu = \omega \cdot \vec{n} = \cos \theta$$

$$\Rightarrow g(\omega) = \frac{\cos \theta}{\int_{H^2} D(\vec{n}_\mu) \max\{0, \omega \cdot \vec{n}_\mu\} d\vec{n}_\mu} = \frac{1}{1 + \Lambda(\omega)}$$

where function Λ can be solved to be:

$$\Lambda(\omega) = \frac{\sqrt{1 + \alpha \tan^2 \theta} - 1}{2}, \left(\alpha = \sqrt{\alpha_x^2 \cos^2 \varphi + \alpha_y^2 \sin^2 \varphi} \right)$$

We use the approximation $G(\omega_1, \omega_2) = \frac{1}{1 + \Lambda(\omega_1) + \Lambda(\omega_2)}$ rather than $G(\omega_1, \omega_2) = g(\omega_1)g(\omega_2)$ since the latter one underestimates and the prior one appears to be more accurate in practice.

2.2.4 Light Sampling

Since rays coming from light sources contribute greatly to a surface's received radiance despite the sparse solid angles they take up, it is of great benefit to deliberately sample rays that point to the light sources rather than relying on rays sampled from the transport model probability distributions.

In this project we implemented power light sampling, which samples rays that point to light sources based on the light's strength:

Power Light Sampling

$$p(\vec{w}_i) = \frac{P_i}{\sum P},$$

where P denotes the power of light i .

Light sampling techniques only have a PDF since the BSDF used should come from the transport models.

2.3 Multiple Importance Sampling (MIS) [1]

2.4 Spectrum

We use the CIE XYZ color space as the source of truth for outputting RGB colors.

To be more specific, the conversion process from radiance from discretely sampled wavelengths to RGB look something like this:

$$\text{RGB (Material)} \rightarrow \lambda(\text{wavelength}) \xrightarrow[\text{Estimator}]{\text{Monte Carlo}} \text{XYZ} \rightarrow \text{RGB}$$

2.5 Summary

Radiance

To sum up the above modeling, the complete rendering equation we simulate looks something like this:

$$\begin{aligned}
 L(\vec{w}_{\text{curr}}, p) &= L_e(p) + \int_{S^2} L(\vec{w}_{\text{next}}, p) f(\vec{w}_{\text{curr}}, \vec{w}_{\text{next}}) \cos\langle \vec{w}_{\text{next}}, \vec{n} \rangle d\vec{w}_{\text{next}} \\
 &= L_e(p) + \sum_{\text{MIS}} \int_{S^2} L(\vec{w}_{\text{next}}, p) f(\vec{w}_{\text{curr}}, \vec{w}_{\text{next}}) \cos\langle \vec{w}_{\text{next}}, \vec{n} \rangle d\vec{w}_{\text{next}} \\
 &\approx L_e(p) + \sum_{\mathcal{J}} \frac{1}{n_{\mathcal{J}}} \sum_{\vec{w}_{\text{next}}} \alpha_{\mathcal{J}}(\vec{w}_{\text{next}}) \frac{\sum_{\mathcal{T}} f_{\mathcal{T}}(\vec{w}_{\text{next}})}{p_{\mathcal{J}}(\vec{w}_{\text{curr}}, \vec{w}_{\text{next}})},
 \end{aligned}$$

where $\mathcal{J} \in \{\text{Sample Model}\} \wedge \mathcal{T} \in \{\text{Transport Model}\}$

Spectrum

And the spectrum part:

$$\begin{aligned}
 X &= \int_{\lambda} \bar{x}(\lambda) L(\lambda) d\lambda \\
 Y &= \int_{\lambda} \bar{y}(\lambda) L(\lambda) d\lambda \\
 Z &= \int_{\lambda} \bar{z}(\lambda) L(\lambda) d\lambda
 \end{aligned}$$

where $\bar{x}, \bar{y}, \bar{z}$ denote the color matching functions:

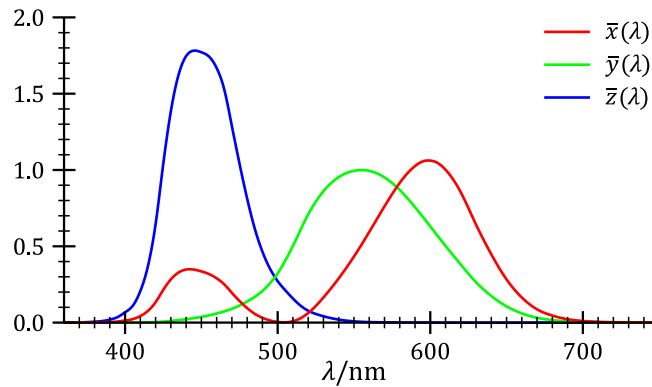


Figure 7: CIE 1931 XYZ Color Matching Functions

Then calculate the RGB values from the XYZ color space:

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 2.36461385 & -0.89654057 & -0.46807328 \\ -0.51516621 & 1.4264081 & 0.0887581 \\ 0.0052037 & -0.01440816 & 1.00920446 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}$$

source: [9]

3 Implementation

3.1 Overview

The pathtracing process is divided into the following steps:

```

0  main:
1      rays = camera.GenerateRays()
2      for ray in rays:
3          rgb = LuminanceToRGB(Render(ray))
4
5  Luminance Render(ray):
6      luminance = 0
7      for i in range(SAMPLE_COUNT):
8          wavelength, prob = SampleWaveLength()
9          luminance += Illuminate(ray) / SAMPLE_COUNT / prob
10     return Luminance(Trace(ray))
11
12 Radiance Trace(ray):
13     # 1. Hit Detection
14     hit_info = ray.Hit(scene)
15     if not hit_info.hit:
16         return EnvironmentRadiance(ray.dir)
17
18     # 2. Sample Ray Generation
19     sample_rays = Sample(ray, hit_info)
20
21     # 3. Sample Tracing (Recursion)
22     sample_radiance = [0]
23     for sample_ray in sample_rays:
24         sample_radiance[sample_ray] = Trace(sample_ray)
25

```

```

26 # 4. Integral Estimation
27 radiance = collect(sample_rays, sample_radiance)
28
29 return radiance

```

3.2 Hit Detection

Hit detection solves the first function in Trace: for a given ray, what is the closest visible surface point? To successfully continue the trace process, we need the following fields from hit detection:

hit	whether the ray intersects any scene triangle
dist	the world-space ray distance to the closest hit
pos	the hit position <code>ray.pos + dist · ray.dir</code>
shading_norm	the interpolated vertex normal transformed to world space
geometry_norm	the triangle face normal transformed to world space
material_idx & uvs	the material attached to the hit primitive & the texture uv coordinate

3.2.1 Bounding Volume Hierarchy (BVH)

The implementation uses a two-level bounding volume hierarchy:

ray → TLAS → instance transform → BLAS → triangle

Both acceleration structures are binary trees built on the CPU using the same simple strategy. For a range of objects, compute the AABB of the whole range and the AABB of the object centers. If the range is small enough, emit a leaf. Otherwise choose the longest center-bounds axis and split the range at the median:

$$\text{axis} = \arg \max_i (u_{b,i} - l_{b,i}),$$

$$\text{mid} = \left\lfloor \frac{l + r}{2} \right\rfloor$$

On the GPU, traversal is iterative using a fixed stack size.

3.2.1.1 Top Level Acceleration Structure (TLAS)

The TLAS represents object instances in world space. For each primitive, the CPU creates:

AABB	the primitive bounds transformed into world space
inv_trans	the inverse world transform used to move rays into primitive-local space
blas_root_idx	the root node for that primitive in the shared BLAS
material_idx	node array the material used when this primitive is hit

During traversal, a ray first tests TLAS node boxes with the slab AABB test. Interior nodes push their two children onto the stack. Leaf nodes transform the ray into local space:

$$\begin{aligned}\text{pos}' &= T^{-1} \begin{pmatrix} \text{pos} \\ 1 \end{pmatrix}, \\ \text{dir}' &= T^{-1} \begin{pmatrix} \text{dir} \\ 0 \end{pmatrix}\end{aligned}$$

Then the BLAS referenced by that instance is traversed. If the BLAS returns a hit closer than the current best distance, the TLAS traversal stores that hit's local normals, UVs, inverse transform, material index, and distance.

After traversal, normals are transformed back to world space using the inverse-transpose transform:

$$n_{\text{world}} = \text{normalize}\left((T^{-1})^T n_{\text{local}}\right)$$

3.2.1.2 Bottom Level Acceleration Structure (BLAS)

The BLAS represents triangles in primitive-local space. Each primitive gets its own BLAS root, but all BLAS nodes and triangle indices are packed into contiguous GPU buffers. Interior BLAS nodes store the index of two adjacent children; leaf nodes store an index offset and a triangle count. Current leaves contain at most two triangles.

For a leaf, the shader tests each triangle with the Moller-Trumbore ray-triangle intersection [10]:

$$\begin{aligned}
e_1 &= v_1 - v_0, \\
e_2 &= v_2 - v_0, \\
h &= d \times e_2, \\
a &= e_1 \cdot h
\end{aligned}$$

If a is near zero, the ray is parallel to the triangle. Otherwise the barycentric coordinates u and v are computed and accepted only when:

$$\begin{aligned}
u &\geq 0, \\
v &\geq 0, \\
u + v &\leq 1, \\
t &> \varepsilon
\end{aligned}$$

The nearest accepted triangle becomes the BLAS hit. The shader interpolates the smooth normal and texture coordinates with the barycentric weights:

$$\begin{aligned}
w_0 &= 1 - u - v, \\
n &= \text{normalize}(w_0 n_0 + u n_1 + v n_2), \\
uv &= w_0 uv_0 + u uv_1 + v uv_2
\end{aligned}$$

It also computes the geometric normal from the triangle edges:

$$n_g = \text{normalize}((v_1 - v_0) \times (v_2 - v_0))$$

The BLAS returns this local hit information to the TLAS, which converts it into the final world-space `HitInfo`.

3.3 Sample Ray Generation

The generation of sampling rays is done with the knowledge of the material of the hit position, as tracing them are **very expensive**. Knowing the material, we can make much higher quality samples with certain probability distributions that reduce noise and make life easier for the `collect` function (the integral estimator).

3.4 GL Transmission Format (glTF)

The glTF format is used as the accepted file format [11]. In order to perform the physics based rendering techniques from Section 1.1, we need to use the material information provided by the file format to determine which scatter

CRYSTAL GRAPHICS DOCUMENT

model or combination of models to use. In addition, we also need to translate the material information to physics metrics to perform precise calculations. It is helpful to think of the process into 2 steps:

glTF material $\rightarrow$ physics properties $\rightarrow$ reflection models

glTF Material Information

- alpha mode
- anisotropy
- sheen
- base color
- clearcoat
- specular
- emission
- dispersion
- transmission
- metallic
- ior
- volume
- roughness
- iridescence

source: [12]

Every glTF color used by the material decoder is reconstructed into a scalar spectral value at that wavelength.

We have this table that shows how physics properties derive from the material information for the sampled wavelength:

Physics Property (Notation)	Formula
Spectral Reflectance ($\rho(\lambda)$)	Smits Reconstruction(base color, λ)
Diffuse Reflectance ($R_{d(\lambda)}$)	$(1 - \text{metallic}) \cdot (1 - \text{transmission}) \cdot \rho(\lambda)$
ior (η)	$\eta(\lambda) = n(\lambda) + k(\lambda)i$, where $\begin{cases} n(\lambda) = \begin{cases} \text{ior} + (\text{ior} - 1) \cdot \frac{\text{dispersion}}{20} \cdot \left(\frac{523655}{\lambda^2} - 1.5168 \right), & \text{dielectric} \\ \text{ior}, & \text{conductor} \end{cases} \\ k(\lambda) = \text{metallic} \cdot \sqrt{\frac{4n(\lambda)}{1 - \rho(\lambda)} - (n(\lambda) + 1)^2} \end{cases}$
Microfacet Roughness ($\alpha = \alpha_x = \alpha_y$)	roughness^2
Transmission (T)	transmission
Emission ($E(\lambda)$)	Smits Reconstruction(emissive, λ)

Since material evaluation happens at one sampled wavelength, the RGB base color is not treated as three independent IOR channels. It

is first converted to $\rho(\lambda)$ using the Smits RGB-to-spectrum reconstruction, and the metallic imaginary IOR is inferred from that spectral reflectance. The approximation is inferred from the assumption that:

- A non-metallic ($\text{metallic} = 0$) material should have the normal non-complex Fresnel reflectance:

$$F_0|_{\text{metallic}=0} = \left(\frac{n-1}{n+1}\right)^2$$

- A fully metallic ($\text{metallic} = 1$) material should have Fresnel reflectance equal to the reconstructed spectral base color:

$$F_0(\lambda)|_{\text{metallic}=1} = \rho(\lambda)$$

Therefore we can use this as the approximation:

$$k(\lambda) = \sqrt{\frac{4n(\lambda)}{1-\rho(\lambda)} - (n(\lambda) + 1)^2},$$

and

$$\eta(\lambda) = n(\lambda) + \text{metallic} \cdot k(\lambda)i$$

Explanation:

$$F_0(\lambda)|_{\text{metallic}=1} = \rho(\lambda) = \left(\frac{\eta(\lambda) - 1}{\eta(\lambda) + 1}\right)^2$$

$$\rho(\lambda) = \left(\frac{n + ki - 1}{n + ki + 1}\right)^2 = \frac{(n-1)^2 + k^2}{(n+1)^2 + k^2}$$

$$1 - \rho(\lambda) = \frac{(n+1)^2 - (n-1)^2}{(n+1)^2 + k^2} = \frac{4n}{(n+1)^2 + k^2}$$

$$k^2 = \frac{4n}{1 - \rho(\lambda)} - (n+1)^2$$

Smits Reconstruction [13]

Suppose $\{c_1, c_2, c_3\} = \{r, g, b\} \wedge c_1 \leq c_2 \leq c_3$,

$$\text{Smits}(\lambda) = c_1 S_{\text{white}(\lambda)} + (c_2 - c_1) S_{c_2 \times c_3}(\lambda) + (c_3 - c_2) S_{c_3}(\lambda)$$

where S_c denote the spectral curve for color c and $c_2 \times c_3$ denote the color you get by mixing c_2 and c_3 . The actual curve data is extracted from the `colour` python package [14].

3.5 Spectrum

The renderer does not trace three RGB channels independently. For each path sample it first chooses one wavelength $\lambda \in [380 \text{ nm}, 720 \text{ nm}]$ using the CIE XYZ matching functions as an importance distribution:

$$p(\lambda) = \frac{\bar{x}(\lambda) + \bar{y}(\lambda) + \bar{z}(\lambda)}{\int_{380}^{720} (\bar{x}(\lambda) + \bar{y}(\lambda) + \bar{z}(\lambda)) d\lambda}$$

We use an approximation of the CIE color matching functions in Figure 7 for computation [15]:

$$\begin{aligned}\bar{x}(\lambda) &= 0.362G_s\left(\lambda; 442.0, \frac{1}{0.0624}, \frac{1}{0.0374}\right) \\ &\quad + 1.056G_s\left(\lambda; 599.8, \frac{1}{0.0264}, \frac{1}{0.0323}\right) \\ &\quad - 0.065G_s\left(\lambda; 501.1, \frac{1}{0.0490}, \frac{1}{0.0382}\right) \\ \bar{y}(\lambda) &= 0.821G_s\left(\lambda; 568.8, \frac{1}{0.0213}, \frac{1}{0.0247}\right) \\ &\quad + 0.286G_s\left(\lambda; 530.9, \frac{1}{0.0613}, \frac{1}{0.0322}\right) \\ \bar{z}(\lambda) &= 1.217G_s\left(\lambda; 437.0, \frac{1}{0.0845}, \frac{1}{0.0278}\right) \\ &\quad + 0.681G_s\left(\lambda; 459.0, \frac{1}{0.0385}, \frac{1}{0.0725}\right)\end{aligned}$$

where

$$G_s(x; \mu, \sigma_1, \sigma_2) = \begin{cases} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma_1}\right)^2} & ; x < \mu \\ e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma_2}\right)^2} & ; x \geq \mu \end{cases}$$

Bibliography

- [1] E. Veach and L. J. Guibas, “Optimally Combining Sampling Techniques for Monte Carlo Rendering,” in *Proceedings of SIGGRAPH 1995*, 1995, pp. 419–428. doi: [10.1145/218380.218498](https://doi.org/10.1145/218380.218498).
- [2] E. Hecht, *Optics*, 5th ed. Pearson, 2017.
- [3] Wikipedia contributors, “Photoreceptor Cell.” Accessed: June 12, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Photoreceptor_cell
- [4] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th ed. MIT Press, 2023. Accessed: June 12, 2026. [Online]. Available: <https://pbr-book.org/4ed/contents>
- [5] J. T. Kajiya, “The Rendering Equation,” in *Proceedings of SIGGRAPH 1986*, 1986, pp. 143–150. doi: [10.1145/15922.15902](https://doi.org/10.1145/15922.15902).
- [6] T. S. Trowbridge and K. P. Reitz, “Average Irregularity Representation of a Rough Surface for Ray Reflection,” *Journal of the Optical Society of America*, vol. 65, no. 5, pp. 531–536, 1975, doi: [10.1364/JOSA.65.000531](https://doi.org/10.1364/JOSA.65.000531).
- [7] B. Walter, S. R. Marschner, H. Li, and K. E. Torrance, “Microfacet Models for Refraction through Rough Surfaces,” in *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, 2007, pp. 195–206. doi: [10.2312/EGWR/EGSR07/195-206](https://doi.org/10.2312/EGWR/EGSR07/195-206).
- [8] B. G. Smith, “Geometrical Shadowing of a Random Rough Surface,” *IEEE Transactions on Antennas and Propagation*, vol. 15, no. 5, pp. 668–671, 1967, doi: [10.1109/TAP.1967.1138991](https://doi.org/10.1109/TAP.1967.1138991).
- [9] Wikipedia contributors, “CIE 1931 Color Space.” Accessed: June 12, 2026. [Online]. Available: https://en.wikipedia.org/wiki/CIE_1931_color_space
- [10] T. Möller and B. Trumbore, “Fast, Minimum Storage Ray/Triangle Intersection,” *Journal of Graphics Tools*, vol. 2, no. 1, pp. 21–28, 1997, doi: [10.1080/10867651.1997.10487468](https://doi.org/10.1080/10867651.1997.10487468).

- [11] Khronos Group, “glTF 2.0 Specification.” Accessed: June 12, 2026.
[Online]. Available: <https://github.com/KhronosGroup/glTF/tree/main/specification/2.0>
- [12] Khronos Group, “Physically-Based Rendering in glTF 2.0.” Accessed: June 12, 2026. [Online]. Available: <https://www.khronos.org/glTF/pbr/>
- [13] B. Smits, “An RGB-to-Spectrum Conversion for Reflectances,”
Journal of Graphics Tools, vol. 4, no. 4, pp. 11–22, 1999, doi:
[10.1080/10867651.1999.10487511](https://doi.org/10.1080/10867651.1999.10487511).
- [14] Colour Developers, “Colour: Colour Science for Python.” Accessed: June 12, 2026. [Online]. Available: <https://www.colour-science.org/>
- [15] C. Wyman, P.-P. Sloan, and P. Shirley, “Simple Analytic Approximations to the CIE XYZ Color Matching Functions,”
Journal of Computer Graphics Techniques, vol. 2, no. 2, pp. 1–11, 2013,
[Online]. Available: <https://jcgt.org/published/0002/02/01/>